

TeamTrack

Project Management Platform for Startups & Teams

Document Type	Product Requirements Document (PRD)
Version	1.0 (Initial Release)
Status	In Development — Backend Phase
Author	Rohit Kumar (@rohitxcodes)
Repository	github.com/rohitxcodes/TeamTrack
Stack	Node.js · Express 5 · MongoDB · JWT · WebSocket
Date	March 14, 2026

Empowering startup teams to collaborate, track, and deliver — seamlessly.

TABLE OF CONTENTS

1. Executive Summary	3
2. Problem Statement & Vision	3
3. Target Audience	4
4. Current Codebase Analysis	4
5. System Architecture	5
6. Data Models & Schemas	5
7. Authentication & Authorization	7
8. API Endpoints — Full Reference	8
9. WebSocket / Real-Time Chat	13
10. Frontend Pages & Routes	14
11. Permission Matrix	16
12. Feature Roadmap	17
13. Non-Functional Requirements	18
14. Error Handling Strategy	18
15. AI / LLM Context Summary	19

1. EXECUTIVE SUMMARY

TeamTrack is a backend-heavy, role-based project-management platform designed to help startup teams collaborate efficiently. Inspired by the workflow of tools such as Jira, TeamTrack provides a lightweight yet powerful alternative purpose-built for small-to-medium teams who need fast onboarding, clear task ownership, and real-time communication — without the overhead of enterprise tooling.

The platform exposes a RESTful JSON API (Node.js + Express 5) backed by MongoDB, with JWT-based stateless authentication stored in secure HTTP-only cookies. A WebSocket layer (Socket.io) will power real-time group and direct messaging. A React (Vite) single-page application consumes the API and delivers a clean, role-aware interface for both Admins and Employees.

2. PROBLEM STATEMENT & PRODUCT VISION

2.1 Problem Statement

Distributed and co-located startup teams frequently struggle with: **task visibility** (who is doing what?), **accountability** (is it done?), and **communication scatter** (information buried in chat threads separate from tasks). Existing tools are either too expensive, too complex to set up, or not designed for the fast-moving nature of early-stage teams.

Pain Point	Impact
No single source-of-truth for task status	Missed deadlines, duplicated work
Employees unaware of assigned tasks until told verbally	Delayed starts, blockers
Admin has no visibility into progress without meetings	Wasted synchronisation overhead
No separation between team tasks and personal to-dos	Context switching & noise
Communication fragmented across WhatsApp / email	Knowledge loss, slow response

2.2 Vision

To be the go-to project-management tool for Indian startups and college project groups — combining task management, role-based access, and team chat in a single, fast, open-source platform.

3. TARGET AUDIENCE

Persona	Description	Key Needs
Startup Founder / Admin	Technical or non-technical founder managing a team of 2-60 people	Create workspace, assign tasks, monitor progress, communicate
Employee / Team Member	Developer, designer, or analyst working within an existing group	See assigned tasks, update status, personal to-dos, message admin
Project Lead (future)	Mid-level manager overseeing a sub-group inside a large organisation	Delegate tasks to specific members, view sub-group reports
Recruiter / Evaluator	Technical recruiter reviewing the project as a portfolio piece	Assess code quality, architecture decisions, API design completeness

4. CURRENT CODEBASE ANALYSIS

4.1 What Is Already Implemented

Module	File(s)	Status	Notes
User Registration	auth.controller.js	✓ Done	bcrypt hashing, duplicate check
User Login	auth.controller.js	✓ Done	JWT in HTTP-only cookie
User Logout	auth.controller.js	✓ Done	Cookie cleared
Auth Middleware	auth.middleware.js	✓ Done	JWT verify, attaches req.user
Admin Guard	admin.middleware.js	✓ Done	Role === 'admin' check
Group-Member Guard	groupedUserOnly.middleware.js	✓ Done	Membership lookup
User Model	User.model.js	✓ Done	name, email, password, role
Task Model	Task.model.js	✓ Done	title, desc, status, createdBy, assignedTo
Group Model	Group.model.js	✓ Done	name, createdBy
Membership Model	Membership.model.js	✓ Done	user, group, role; unique index
Admin Create Task	admin.controller.js	■ Partial	Creates task but no assignedTo
Admin Routes	admin.routes.js	■ Stub	PATCH/DELETE/GET users are stubs
Group Routes	group.routes.js	■ Stub	All handlers return placeholder text
Task Controller	task.controller.js	✗ Empty	File exists but no logic
DB Connection	config/db.js	✓ Done	Mongoose connect
App Bootstrap	app.js + index.js	✓ Done	Express, cookie-parser, routes mounted

4.2 What Needs to Be Built

- Complete task assignment flow (assignedTo + group scoping)
- All group route controller logic (invite, members CRUD, task CRUD within group)
- Personal tasks — scoped to the employee only, hidden from admin
- Employee task status update (PATCH /groups/:id/tasks/:id/status)
- Chat system — Message model, Socket.io server, REST history endpoints
- Invite system — email-based or token-based invite for employees
- Frontend (React + Vite) — all pages described in Section 10
- Input validation layer (Zod / express-validator)
- Centralised error-handling middleware
- Environment-based config (development / production)
- Pagination for list endpoints

- Role promotion/demotion endpoint

5. SYSTEM ARCHITECTURE

Layer	Technology	Responsibility
HTTP Server	Node.js 20 LTS + Express 5	Route handling, middleware pipeline
Real-Time	Socket.io 4	WebSocket upgrade, room management, message broadcast
Database	MongoDB Atlas + Mongoose 9	Persistence, schema validation, indexing
Auth	JWT (jsonwebtoken) + bcrypt	Stateless authentication, password hashing
Session	HTTP-only cookie	Token transport, CSRF mitigation
Frontend	React 18 + Vite + Axios	SPA, role-aware routing, real-time chat
Validation	Zod (planned)	Request body schema validation
Deployment	Railway / Render (planned)	Backend; Vercel / Netlify for frontend

Request lifecycle: Client → CORS → cookie-parser → JSON parser → Route → Auth Middleware → Role Middleware → Controller → Mongoose → MongoDB → JSON Response. WebSocket connections upgrade at /socket.io and are authenticated via the same JWT cookie on the handshake.

6. DATA MODELS & SCHEMAS

6.1 User

Field	Type	Required	Constraints	Notes
_id	ObjectId	auto	unique	MongoDB primary key
name	String	Yes	trim, maxLen 80	Full name
email	String	Yes	unique, lowercase	Login identifier
password	String	Yes	bcrypt hash, minLen 8	Never returned in API
role	String	Yes	enum: admin employee	Default: employee
createdAt	Date	auto	—	Timestamp
updatedAt	Date	auto	—	Timestamp

6.2 Group

Field	Type	Required	Notes
_id	ObjectId	auto	Primary key
name	String	Yes	e.g. 'Backend Squad', trim, maxLen 100

Field	Type	Required	Notes
createdBy	ObjectId	Yes	Ref: User (admin who created the group)
createdAt	Date	auto	Timestamp
updatedAt	Date	auto	Timestamp

6.3 Membership

Field	Type	Required	Notes
_id	ObjectId	auto	Primary key
user	ObjectId	Yes	Ref: User
group	ObjectId	Yes	Ref: Group
role	String	Yes	enum: admin member — default: member
createdAt	Date	auto	Timestamp

Unique compound index on (user, group) prevents duplicate memberships.

6.4 Task

Field	Type	Required	Notes
_id	ObjectId	auto	Primary key
title	String	Yes	maxLen 200
description	String	No	maxLen 2000
status	String	Yes	enum: pending in-progress completed — default: pending
createdBy	ObjectId	Yes	Ref: User (admin or self for personal tasks)
assignedTo	ObjectId	No	Ref: User — null for personal tasks
group	ObjectId	No	Ref: Group — null for personal tasks
isPersonal	Boolean	No	true = personal task, hidden from admin
priority	String	No	enum: low medium high — default: medium (planned)
dueDate	Date	No	Optional deadline (planned)
createdAt	Date	auto	Timestamp
updatedAt	Date	auto	Timestamp

6.5 Message (NEW — required for Chat)

Field	Type	Required	Notes
_id	ObjectId	auto	Primary key

Field	Type	Required	Notes
sender	ObjectId	Yes	Ref: User
group	ObjectId	No	Ref: Group — set for group messages
recipient	ObjectId	No	Ref: User — set for DMs
content	String	Yes	maxLen 4000
readBy	[ObjectId]	No	Array of User refs who have read the message
createdAt	Date	auto	Used for message ordering

Index on (group, createdAt) and (sender, recipient, createdAt) for efficient pagination.

6.6 Invite (NEW — required for Invite Flow)

Field	Type	Required	Notes
_id	ObjectId	auto	Primary key
email	String	Yes	Invitee email
group	ObjectId	Yes	Ref: Group
token	String	Yes	UUID v4 — single-use
status	String	Yes	enum: pending accepted expired
expiresAt	Date	Yes	Default: 7 days from creation
createdBy	ObjectId	Yes	Ref: User (admin)
createdAt	Date	auto	Timestamp

7. AUTHENTICATION & AUTHORIZATION

7.1 Authentication Flow

Step	Description
Register	POST /api/auth/register → validate body → hash password (bcrypt, 10 rounds) → save User
Login	POST /api/auth/login → verify password → sign JWT {id, role} with JWT_SECRET → set HttpOnly cookie 'token' (maxAge 7d)
Session Check	Every protected route → auth.middleware extracts & verifies cookie JWT → attaches req.user
Logout	POST /api/auth/logout → res.clearCookie('token')

7.2 Middleware Chain

Middleware	File	Purpose	Applied To
restrictedUserOnly	auth.middleware.js	Validates JWT cookie; populates req.user	All protected routes

Middleware	File	Purpose	Applied To
adminOnly	admin.middleware.js	Ensures req.user.role === 'admin'	Admin-only routes
groupedUserOnly	groupedUserOnly.middleware.js	Validates active Membership for req.user in :group	Group-scoped routes
validateBody (planned)	validate.middleware.js	Zod schema validation on req.body	POST / PATCH routes
errorHandler (planned)	error.middleware.js	Centralised error format	Global (app.use last)

7.3 JWT Payload

```
{ "id": "<userId>", "iat": <timestamp> }
```

The role is **not** stored in the JWT; it is always fetched fresh from the database to prevent stale-role attacks.

8. API ENDPOINTS — FULL REFERENCE

Base URL: **https://api.teamtrack.app/api** (local: http://localhost:5000/api). All endpoints return **application/json**. HTTP status codes follow REST conventions. Auth = JWT cookie. Admin = role admin in current group.

8.1 Authentication — /api/auth

Method	Endpoint	Auth	Body / Params	Response	Description
POST	/register	None	name, email, password, role	201 {message}	Register new user account
POST	/login	None	email, password	200 {message} + Set-Cookie	Login; sets JWT cookie
POST	/logout	JWT	—	200 {message}	Clears auth cookie
GET	/me	JWT	—	200 {user}	Returns current user profile
PATCH	/me	JWT	name?, email?, password?	200 {user}	Update own profile (planned)

8.2 Admin — /api/admin (Admin only)

Method	Endpoint	Body / Params	Response	Description
POST	/tasks	title, description, status, assignedTo, groupId	201 {task}	Create and assign task to employee
GET	/tasks	?groupId=&status=&assignedTo=&page=&limit=	200 [{task}]	List all tasks (with filters, pagination)
GET	/tasks/:id	—	200 {task}	Get single task detail
PATCH	/tasks/:id	title?, description?, status?, assignedTo?	200 {task}	Update any task field
DELETE	/tasks/:id	—	200 {message}	Delete task
GET	/users	?page=&limit=&role=	200 [{user}]	List all users (no passwords)
GET	/users/:id/tasks	?status=	200 [{task}]	All tasks for a specific employee
DELETE	/users/:id	—	200 {message}	Remove user account

8.3 Groups — /api/groups

Method	Endpoint	Auth	Body / Params	Response	Description
POST	/	JWT	name	201 {group}	Create new group (creator becomes admin)
GET	/my	JWT	—	200 [{group}]	All groups the current user belongs to
GET	/:groupId	Member	—	200 {group, members}	Group details + member list
DELETE	/:groupId	Admin	—	200 {message}	Delete group and all related data (planned)
POST	/:groupId/invite	Admin	email	200 {invite}	Send invite (email or token link)
POST	/:groupId/join	JWT	token	200 {membership}	Accept invite token
GET	/:groupId/members	Admin	—	200 [{membership}]	List all group members with roles
PATCH	/:groupId/members/:userId/role	Admin	role (admin member)	200 {membership}	Promote or demote a member
DELETE	/:groupId/members/:userId	Admin	—	200 {message}	Remove member from group

8.4 Group Tasks — /api/groups/:groupId/tasks

Method	Endpoint	Auth	Body / Params	Response	Description
POST	/	Admin	title, desc, assignedTo, priority?, dueDate?	201 {task}	Admin creates & assigns task
GET	/	Admin	?status=&assignedTo=&page=&limit=	200 [{task}]	Admin sees ALL tasks in group
GET	/my	Member	?status=	200 [{task}]	Employee sees only their assigned tasks
GET	/:taskId	Member	—	200 {task}	Get task (employee sees own; admin sees any)
PATCH	/:taskId/status	Member	status (in-progress completed)	200 {task}	Employee updates their task status
PATCH	/:taskId	Admin	title?, desc?, assignedTo?, priority?, dueDate?	200 {task}	Admin edits any task field
DELETE	/:taskId	Admin	—	200 {message}	Admin deletes task

8.5 Personal Tasks — /api/tasks/personal (Employee only — hidden from Admin)

Method	Endpoint	Body / Params	Response	Description
POST	/	title, description?, priority?	201 {task}	Create personal task
GET	/	?status=	200 [{task}]	List own personal tasks
PATCH	/:taskId	title?, description?, status?	200 {task}	Update personal task
DELETE	/:taskId	—	200 {message}	Delete personal task

8.6 Chat (REST history) — /api/chat

Method	Endpoint	Auth	Params	Description
GET	/groups/:groupId/messages	Member	?before=&limit=	Paginated group message history

Method	Endpoint	Auth	Params	Description
GET	/dm/:userId/messages	JWT	?before=&limit=	Paginated DM history with user
DELETE	/messages/:messageId	Sender	—	Delete own message
GET	/dm/conversations	JWT	—	List all DM conversations (last message)

9. WEBSOCKET / REAL-TIME CHAT

9.1 Connection

Client connects to `ws://api.teamtrack.app` via Socket.io. The JWT cookie is validated on the **connection** handshake; unauthenticated sockets are disconnected immediately.

9.2 Rooms

Room Pattern	Members	Purpose
group:{groupId}	All members of the group	Group broadcast messages
dm:{sorted_userId_pair}	Two specific users	Direct messages
user:{userId}	Single user	Notifications, read receipts

9.3 Socket Events

Event (emit / on)	Direction	Payload	Description
join_group	Client → Server	{groupId}	Join group room
leave_group	Client → Server	{groupId}	Leave group room
group_message	Client → Server	{groupId, content}	Send group message
group_message	Server → Room	{message object}	Broadcast to room
dm_message	Client → Server	{recipientId, content}	Send direct message
dm_message	Server → DM Room	{message object}	Deliver to DM room
typing	Client → Server	{groupId?, recipientId?}	Typing indicator
typing	Server → Room	{userId, name}	Forward indicator
message_read	Client → Server	{messageId}	Mark message read
disconnect	Server internal	—	Clean up rooms

10. FRONTEND — PAGES & ROUTES

Stack: **React 18 + Vite + React Router v6 + Axios + Socket.io-client + TailwindCSS**. The frontend is a SPA with role-aware routing protected by a PrivateRoute component that checks session state before rendering.

10.1 Public Routes (no auth required)

Path	Component	Description
/	LandingPage	Marketing / hero page with CTA to sign up or log in
/login	LoginPage	Email + password form; redirects to /dashboard on success
/register	RegisterPage	Name, email, password, role selector; redirects to /dashboard
/invite/:token	InvitePage	Accepts invite token; prompts login/register then joins group

10.2 Authenticated Routes — Employee

Path	Component	Description
/dashboard	EmployeeDashboard	Summary: pending tasks count, personal to-dos, recent messages
/groups	MyGroupsList	Cards for each group the user belongs to
/groups/:groupId	GroupOverview	Group detail: members list, task summary
/groups/:groupId/tasks	MyTaskList	Employee's assigned tasks in this group; filter by status
/groups/:groupId/tasks/:taskId	TaskDetail	Full task view; status update button
/groups/:groupId/chat	GroupChat	Real-time group chat panel
/messages	DMInbox	All DM conversations list
/messages/:userId	DMConversation	Direct message thread with admin or peer
/personal-tasks	PersonalTasks	Private to-do list; CRUD; hidden from admin
/profile	ProfilePage	View & edit own name, email, password

10.3 Authenticated Routes — Admin (additional)

Path	Component	Description
/admin/dashboard	AdminDashboard	Summary cards: total employees, tasks by status, active groups
/admin/groups	AdminGroupList	All groups created by admin; create-group button
/admin/groups/new	CreateGroupForm	Form: group name → creates group
/admin/groups/:groupId	AdminGroupDetail	Members list, all tasks in group, invite button
/admin/groups/:groupId/tasks/new	CreateTaskForm	Form: title, desc, assignee (dropdown), priority, due date
/admin/groups/:groupId/tasks	AdminTaskBoard	Kanban/list view of ALL group tasks; filter by employee/status
/admin/groups/:groupId/members	MemberManagement	Role toggle (admin/member), remove member
/admin/users	UserDirectory	All registered users; view task load; remove user
/admin/users/:userId/tasks	UserTaskView	All tasks for a specific employee
/admin/groups/:groupId/chat	AdminGroupChat	Group chat (admin participates)

Path	Component	Description
/admin/messages/:userId	AdminDMThread	DM with individual employee

10.4 Component Architecture (shared)

Component	Used In	Purpose
Navbar	All auth pages	Logo, nav links, logout button, notification bell
Sidebar	Dashboard layouts	Navigation menu (role-aware links)
TaskCard	TaskList, Board	Task summary with status badge, assignee avatar
StatusBadge	TaskCard, TaskDetail	Coloured pill: pending / in-progress / completed
MemberAvatar	GroupDetail, Chat	User avatar with name tooltip
ChatPanel	GroupChat, DMConversation	Socket.io-powered message list + input
PrivateRoute	Router	Redirects unauthenticated users to /login
RoleRoute	Router	Restricts pages to admin or employee role
Notification	Navbar	Toast notifications for new messages, task updates

11. PERMISSION MATRIX

Action	Unauthenticated	Employee (non-member)	Group Member	Group Admin	Platform Admin
Register / Login	✓	—	—	—	—
View own profile	x	✓	✓	✓	✓
Create group	x	✓	✓	✓	✓
View group details	x	x	✓	✓	✓
Invite member to group	x	x	x	✓	✓
Remove member from group	x	x	x	✓	✓
Create task (group)	x	x	x	✓	✓
View ALL group tasks	x	x	x	✓	✓
View OWN tasks only	x	x	✓	✓	✓
Update task status	x	x	Own tasks	✓	✓
Edit / Delete any task	x	x	x	✓	✓
Create personal task	x	✓	✓	✓	✓
View others' personal tasks	x	x	x	x	x
Send group chat message	x	x	✓	✓	✓
Send DM	x	✓	✓	✓	✓

Action	Unauthenticated	Employee (non-member)	Group Member	Group Admin	Platform Admin
Delete own message	✗	✓	✓	✓	✓
Delete any message	✗	✗	✗	✓	✓

12. FEATURE ROADMAP

Phase	Milestone	Features	Target
v0.1 (Current)	Backend Skeleton	Auth, User & Task models, Admin routes (stubs), Group model, Member model	Scope Middleware
v0.2	Core API	Complete group CRUD, task assignment, employee task status update, Personal tasks, input validation	Scope 1
v0.3	Chat & Invites	Message model, Socket.io server, group chat, DMS, invite token system, Spatial notification	Scope 2
v0.4	Frontend MVP	React SPA, auth flow, dashboard, group pages, task board, basic chat UI	Sprint 3
v0.5	Polish & Deploy	Error handling, pagination, loading states, responsive design, Railway + Supabase	Sprint 4 Deploy
v1.0	Release	File attachments (tasks), task comments, admin analytics dashboard, dashboard, PWA	Scope 3

13. NON-FUNCTIONAL REQUIREMENTS

Category	Requirement	Target Metric
Performance	API response time	< 300 ms for 95th percentile (local)
Scalability	Concurrent WebSocket connections	Supports 200+ connections (Node.js cluster)
Security	Password storage	bcrypt, cost factor 10+
Security	JWT secret	Min 32-char random string; stored in .env
Security	Cookie flags	HttpOnly, SameSite=Strict, Secure (prod)
Security	Input validation	All POST/PATCH bodies validated via Zod
Reliability	Error handling	Global middleware; all async wrapped in try/catch
Maintainability	Code structure	MVC pattern; controllers separate from routes
Observability	Logging	Morgan HTTP logger + winston (planned)
Portability	Environment config	All secrets in .env; .env.example committed

14. ERROR HANDLING STRATEGY

All API responses must follow a uniform JSON shape to enable consistent frontend error handling:

```
{ "success": false, "message": "Human-readable description", "code": "ERROR_CODE" }
```

HTTP Status	Error Code	When Used
400 Bad Request	VALIDATION_ERROR	Missing/invalid request body fields

HTTP Status	Error Code	When Used
401 Unauthorized	NOT_AUTHENTICATED	No or invalid JWT cookie
403 Forbidden	ACCESS_DENIED	Valid JWT but insufficient role/membership
404 Not Found	RESOURCE_NOT_FOUND	Task, Group, User ID does not exist
409 Conflict	DUPLICATE_RESOURCE	Duplicate email on register, duplicate membership
500 Internal Server	SERVER_ERROR	Unexpected exception; logged server-side

15. AI / LLM CONTEXT SUMMARY

This section provides a condensed context block that can be pasted directly into any AI assistant (Claude, GPT-4, Gemini, etc.) to quickly orient it to the codebase.

PROJECT: TeamTrack — Node.js/Express 5 + MongoDB + JWT + Socket.io project management API
AUTHOR: Rohit Kumar (@rohitxcodes) | GITHUB: github.com/rohitxcodes/TeamTrack | STATUS: In Development

STACK: Node.js 20, Express 5, Mongoose 9, bcrypt, jsonwebtoken, cookie-parser, Socket.io (planned),
React 18 + Vite + React Router v6 + TailwindCSS (planned frontend)

MODELS: User {name, email, password(hash), role(admin|employee)}
Group {name, createdBy}
Membership {user, group, role(admin|member)} — unique index on (user,group)
Task {title, description, status(pending|in-progress|completed), createdBy, assignedTo, group, isPersonal, priority, dueDate}
Message {sender, group?, recipient?, content, readBy, createdAt} ← TO BE BUILT
Invite {email, group, token, status, expiresAt, createdBy} ← TO BE BUILT

AUTH: JWT stored in HttpOnly cookie 'token'. Middleware: restrictedUserOnly (all protected routes),
adminOnly (admin-only actions), groupedUserOnly (validates active Membership for :groupId).

IMPLEMENTED: register, login, logout, adminCreateTask (partial), all middleware, all models,
route stubs for admin CRUD and group CRUD.

INCOMPLETE (stubs returning placeholder strings): all group controller logic, task assignment,
employee status update, personal tasks, chat, invite system.

KEY RULES:

- Employees can ONLY see their OWN assigned tasks (never other employees' tasks)
- Personal tasks (isPersonal:true) are invisible to admin and all other users
- Admin in group context = Membership.role === 'admin' (not User.role)
- Chat: group messages broadcast via Socket.io room 'group:{groupId}'; DMs via 'dm:{sortedPair}'

ROUTES MOUNTED: /api/auth, /api/admin (admin-only), /api/groups (group CRUD + tasks + chat)

```
PERSONAL TASKS ROUTE: /api/tasks/personal (to be created)
CHAT REST ROUTE: /api/chat (to be created)
```

```
CURRENT GAPS TO FILL: group controller logic, task controller, personal task
route+controller,
chat REST + Socket.io server, invite system, Zod validation, error middleware, pagination.
```

How to use this context: Copy the block above and prepend it to any prompt you give to an AI assistant. For example: *'Given this project context, implement the groupedUserOnly middleware controller logic for POST /api/groups/:groupId/tasks.'*